```
        <li><a href="/2">Listing 7.2</a></li>
        <li><a href="/3">Listing 7.3</a></li>
        <li><a href="/4">Listing 7.4</a></li>
      </ul>
    '''
  handler res

script = (res, listing) ->
  res.writeHead 200, 'Content-Type': 'application/javascript'
  fs.readFile "7.#{listing}.coffee", 'utf-8', (e, source) ->
    if e then res.end "/* #{e} */"
    else res.end coffee.compile source

server.listen 8080, '127.0.0.1'
```

So, what else did you zoom through in earlier chapters? Comprehensions! It's now time to revisit comprehensions and explore their use.

7.5. When to use comprehensions (and when not to)

Comprehensions provide a natural and powerful syntax for dealing with elements in arrays and properties of objects. You'll recognize this simple comprehension for even numbers:

```
evens = (num for num in [1..10] when num%2 == 0)
```

A comprehension is much easier to read than a JavaScript `for` loop:

```
numbers = [1,2,3,4,5,6,7,8,9,10]
evens = []


for (var i = 0; i !== numbers.length; i++) {
  if(numbers[i]%2 === 0) {
  evens.push(numbers[i]);
}
```

That's not really a fair comparison with JavaScript, though. In recent years it has become more common in JavaScript to use array methods such as `map`, `reduce`, and `filter` instead of `for` loops:

```
evens = numbers.filter(function(item) {
  return item%2 == 0;
```